

```

satya@Elegance: ~/Blog/ForExample
$ ls -l for_without_volatile.s
-rw-rw-r-- 1 satya satya 490 Oct 24 18:26 for_without_volatile.s
satya

satya@Elegance: ~/Blog/ForExample
$ ls -l for_with_volatile.s
-rw-rw-r-- 1 satya satya 542 Oct 24 18:26 for_with_volatile.s
satya

```

Figure 4: Size of assembly codes generated by the compiler with and without volatile qualifier

the infinite loop labelled `.L2`.

In contrast, in the disassembly of the volatile version (`while_loop_with_volatile.s`) of the `while` loop shown in Figure 3, the compiler assumes that the value of the flag variable can change outside the program and performs no optimisations. Consequently, the value of the flag is loaded into the register `%eax` every time from the memory `[-8(%ebp)]` inside the loop labelled `.L2`. As a result, the value of the flag is checked every time, and further decisions are taken depending upon the value of the flag variable.

To avoid optimisation problems caused by changes to the program state external to the implementation, it is always safer to declare the variable as 'volatile'. This helps to avoid unexpected results. From Figure 3, we can conclude that the 'volatile' key word prevents optimisation of the code by the compiler.

Illustration 2: Delay generations using loops

Let us consider another example, where 'for' loops are used commonly in the Embedded C code to generate small delays as shown in the following code:

```

1 int main()
2 {
3     int i;
4
5     //Some code goes here
6
7     //Loop for delay generation
8     for( i = 0; i < 100; i++)
9     {
10        ;
11    }
12
13    //Again the remaining code goes here
14
15    return 0;
16 }

```

In fact, a compiler might optimise the code shown above into nothing. A local variable 'i' is the counter for a loop that does nothing but increment value 'i' until it's equal to 100. Thus, the optimiser can replace the loop with a single assignment that just sets 'i' to its final value. When that happens, the delay code doesn't achieve what the programmer had intended. So, it is

always better to declare the local variable 'i' as 'volatile' even though the code might be less efficient, since we will get the desired results, as shown in the code below:

```

1 int main()
2 {
3     volatile int i;
4
5     //Some code goes here
6
7     //Loop for delay generation
8     for( i = 0; i < 100; i++)
9     {
10        ;
11    }
12
13    //Again the remaining code goes here
14
15    return 0;
16 }

```

Let us generate the assembly codes for the examples given in the last two code snippets above, using the commands given in 'How to generate assembly code from the C source code in the gcc compiler', and then get the size of the assembly codes using the `ls` command as given in Figure 4.

Comparing the sizes in Figure 4, one can conclude that the compiler is applying the optimisation techniques without the volatile qualifier. In other words, the compiler is not allowed to reorder the instructions on volatile variables with respect to other memory operations. The disassembly code, both with and without the 'volatile' key word, for Illustration 2 is shown in Figure 5.

Illustration 3: Global variables accessed by multiple tasks within a multi-threaded application

Let us consider one more example to show how the global variable will be affected by the compiler optimisation in a multi-threaded application. The example code snippet is shown below:

```

1 #define FALSE 0
2 #define TRUE 1
3
4 volatile unsigned int global_item_count;
5
6 //Other functions
7 void thread_one(void)
8 {
9     global_item_count = FALSE;
10    while(global_item_count == FALSE)
11    {
12        sleep(1);
13    }

```